Metric-Triggered WSD Decay for Language Model Pretraining

Pierre Bernadet, Georges-Alex Nahas
CS-439 Optimization for Machine Learning, EPFL, Spring 2026

***Abstract*—Warmup-Stable-Decay (WSD) learning rate schedules have emerged as a flexible alternative to cosine scheduling for language model pretraining, decoupling the stable training phase from a short and sharp decay. Yet the key hyperparameters governing this decay: timing, duration, and final learning rate ratio, remain largely empirical [1], and recommendations tuned for specific model and data scales do not automatically transfer to new settings. In this work, we study the relationship between online training metrics and the optimal decay trigger in WSD, using a LLaMA architecture model trained on FineWeb. We reproduce scheduler comparison results from prior work, analyze how decay timing and magnitude affect final validation loss, and propose adaptive metric-based policies to automatically identify the "sweet spot" for initiating decay.**

I. Introduction

Cosine learning rate scheduler [2] has long been the default schedule for pretraining large language models [3], but it imposes a hard token budget: extending a run invalidates the schedule and requires retraining from scratch to maintain optimality. WSD schedules [4] address this limitation by separating training into three phases: a warmup, a long stable phase at peak learning rate, and a short learning rate (LR) decay. Since the decay is applied from a checkpoint at the end of the stable phase, training can be extended simply by continuing the stable phase from this checkpoint and triggering a new decay branch later, without modifying or restarting the earlier training. Figure 1 depicts the differences between the two schedulers.

However, WSD comes with its own set of design choices. When should decay begin? How long should it last, and how deep should the learning rate drop? Existing work [4], [5] provides recommendations that are largely empirical and tuned for specific model and data scales. Finding the timing yielding the best final validation loss, what we refer to as the *sweet spot*, is non-trivial, and running a large number of decay branches to search for it quickly becomes computationally expensive.

Our work addresses three questions:

1) Can we reproduce the WSD vs. cosine comparison from prior work [5] on our model and dataset?
2) Do online training metrics correlate with validation loss under different decay timings and magnitudes, allowing us to predict the *sweet spot*?
3) Can we construct meaningful adaptive policies, based on these metrics, that improve fixed-schedule baselines?

II. Related Work

Learning rate scheduling is a critical and sensitive component of LLM pretraining [3], [6]. Cosine scheduling has been the dominant choice [7], but it requires committing to a total token budget upfront, limiting training flexibility [8]. To overcome this rigidity, *Hu et al.* [4] introduced the WSD scheduler as part of the MiniCPM training framework. *Wen et al.* [5] provided a theoretical grounding for WSD through the *river valley* loss landscape perspective: the stable phase navigates the model along a valley where SGD noise keeps iterates away from sharp walls, and decay silences the noise to let the model descend to the valley floor. Their analysis also introduced WSD-Simplified (WSD-S), demonstrating faster convergence per token than cosine scheduling. *Hägele et al.* [1] further studied WSD at scale, concluding that no single decay recipe is universally optimal. More broadly, adaptive learning rate methods have explored automatic schedule adjustment from training signals, though primarily in the context of per-parameter scaling rather than global schedule timing [9]. All of this prior work treats the timing and magnitude of the decay as fixed with hand-tuned hyperparameters, and no prior work adapts the global decay trigger from online training signals. Our study fills this gap by asking whether *online* training signals can replace these manual choices.

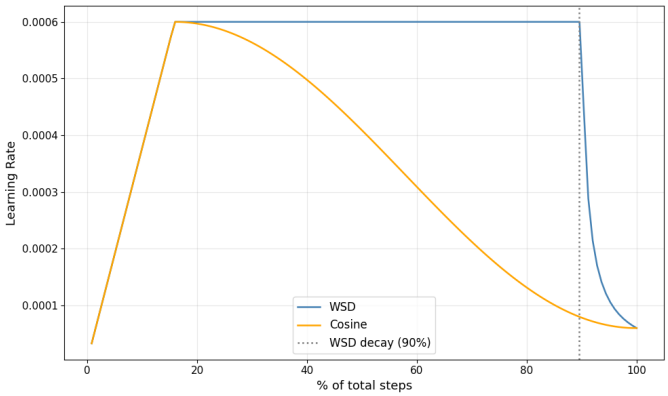

Fig. 1: Comparison of cosine and WSD schedulers with warmup

III. Materials and Methods

A. Dataset

We use FineWeb [10] (`sample-100BT`, 100B tokens) as our pretraining corpus, a large-scale, high-quality web crawl widely adopted in recent open LLM research. FineWeb does not dispose of a dedicated validation split [11]; we therefore stream the training data with a fixed token offset to keep a

held-out validation set, which remains consistent across all experiments.

B. Model

We train a LLaMA architecture [7] causal language model with 8 transformer layers, hidden dimension 512, intermediate size 2048, and 8 attention heads, yielding approximately 40M parameters. This size provides an acceptable trade-off between LLM representativeness, GPU computation limits, and the constraint of running many ablation experiments. All experiments were run on a single NVIDIA H100 GPU.

We use the GPT-2 tokenizer (vocabulary size 50,257) and a context length of 1024 tokens. We follow the optimization setup of *Wen et al.* [5], using AdamW [12] with $\beta_1 = 0.9$, $\beta_2 = 0.95$, weight decay 0.1, a peak learning rate of 6×10^{-4}, and gradient clipping at norm 1.0. While the river valley theory [5] is developed under SGD, AdamW remains the standard optimizer for LLM pretraining in practice. Warmup lasts the first 100 steps. Effective batch size is 524,288 tokens (batch 8, accumulation 64, context 1024), giving a total token budget of 5.24B tokens over 10,000 steps. We use 5,000 validation samples sampled from a held-out stream of FineWeb, evaluated every 50 steps.

C. Experimental Setup

All experiments share the base configuration of Section III-B unless stated otherwise. Training metrics are logged every 5 steps. We design four experiments going from reproduction to adaptive policies.

1) Experiment 1 — Scheduler Comparison: Reproduce the cosine vs. WSD comparison of [5] on our model. A single WSD run (decay starting at 90% of steps, lasting 10%, final LR ratio 0.1) is compared against a cosine run with identical peak LR and training budget, following the paper's recommended hyperparameters. This aims to confirm that WSD achieves comparable performance to the cosine scheduler.

2) Experiment 2 — AdamW vs. Preconditioned-SGD Decay: The theoretical analysis of [5] assumes SGD, yet we train with AdamW whose first-moment estimate accumulates gradient history from the stable phase, potentially conflicting with the sharp descent the river valley theory prescribes. We therefore shrink $\beta_1 \to 0$ and move $\beta_2 \to 1$ during decay, yielding a momentum-free, diagonally preconditioned SGD-like update that we call WSD-PSGD (see Appendix A for the derivation). We compare three conditions: (a) WSD with standard AdamW, (b) cosine with standard AdamW, and (c) WSD-PSGD, i.e. WSD with AdamW betas annealed during decay.

3) Experiment 3 — Decay Timing and Amount Sweep: To map how the *sweet spot* depends on when and how strongly the decay is applied, we run a 5,000-step grid of WSD runs varying the decay start step (as a fraction of total training: 50%, 60%, 70%, 80%, 90%) simultaneously with the final LR ratio ($r \in \{0.5, 0.3, 0.2, 0.1, 0.03\}$). For each configuration we record both the validation loss and 10 online diagnostic metrics throughout training (see Appendix B for the full list and definitions). The goal is to find reliable correlations between these pre-decay metrics and post-decay performance.

4) Experiment 4 — Adaptive Decay Policies: Based on the correlations found in Experiment III-C3, we propose three adaptive policies that monitor training metrics in real time and trigger decay automatically once the metric indicates the *sweet spot* has been reached. This experiment runs for 7,000 steps.

Three design choices are shared across all policies. (i) **Gate** (70% of total steps): the gate enforces a minimum stable-phase duration, ensuring that policies can only fire once training has entered a plateau phase. Metric changes during a plateau are more reliable indicators of a genuine regime shift than those observed during an active loss decrease, making the trigger more meaningful. (ii) **Percentile threshold** (3rd percentile): the policy's decay fires when the monitored metric drops below the 3rd percentile of its lookback distribution, flagging an unusually low value as a potential sweet spot signal. (iii) **Lookback window** (10% of total steps before the gate): percentile thresholds are estimated over this window, which is large enough for stable percentile estimation (Appendix C) yet short enough to reflect the current training regime.

The three policies are:

- **Loss-variance policy**: triggers when `loss_variance` falls below the 3rd percentile of its lookback distribution.
- **Grad-SNR policy**: triggers when `grad_snr` falls below the 3rd percentile of its lookback distribution.
- **Combined policy**: requires *both* conditions simultaneously.

Decay length is total steps minus decay start. We compare each policy against a fixed baseline that starts decay at 90% of steps as recommended by [5].

IV. Results

Exp. 1 & 2 (Figure 2): WSD beats cosine scheduling, with a sharp validation loss drop as decay begins (vertical dotted line), consistent with prior work [5]. Shrinking AdamW betas to approximate preconditioned SGD (WSD-PSGD) does not yield improvement over standard AdamW (final loss worse by 1.3%).

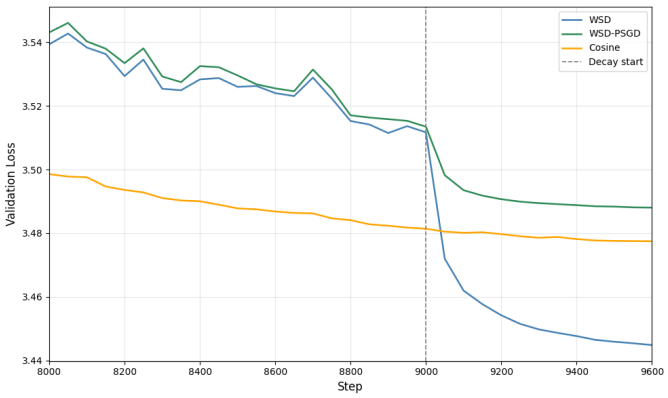

Fig. 2: Cosine vs. WSD vs. WSD-PSGD validation loss on FineWeb (10k steps). Decay starts at step 9,000 (90%) for WSD runs.

Exp. 3 (Figure 3): The decay advantage is defined as the after-decay minus pre-decay validation loss (more negative indicates

greater benefit). A consistent trend appears across all final LR ratios: lower pre-decay gradient SNR correlates with larger decay advantage.

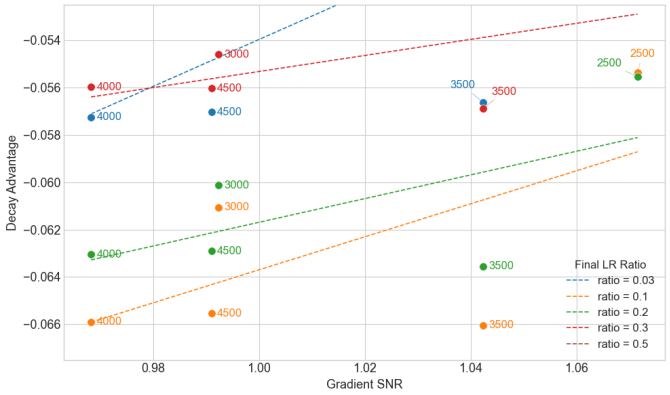

Fig. 3: Decay advantage vs. pre-decay gradient SNR, for each combination of decay start step (point labels) and final LR ratio (color). Dashed lines show per-ratio linear trends. FineWeb, 5k steps total.

Exp. 4 (Figure 4): Adaptive policies trigger decay broadly in the 80%–90% range and achieve validation losses similar with the 90% fixed-decay baseline (final loss better by 0.03%–0.13%).

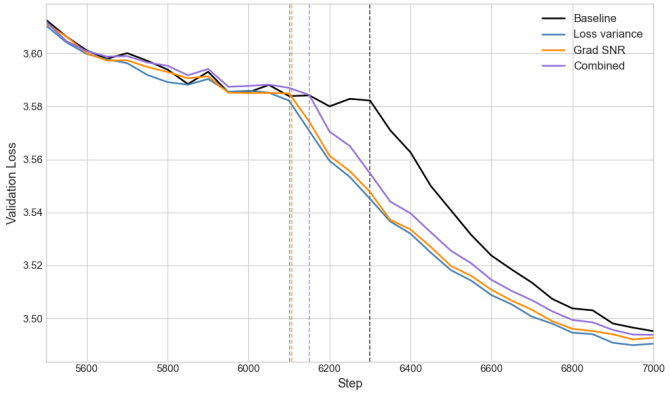

Fig. 4: Validation loss for adaptive policies (loss variance, grad SNR, combined) vs. fixed baseline at 90% trigger. FineWeb, 7k steps total, WSD baseline.

V. Discussion

The experiments confirm that WSD is a strong alternative to cosine scheduling, while also showing that automatically choosing the decay point from online metrics remains challenging. In Section III-C1, WSD reproduces the sharp validation-loss drop during decay described by Wen et al. [5] (Figure 2). However, Section III-C4 shows that converting this intuition (Figure 3) of an automatic trigger is not straightforward. The adaptive policies identify suitable decay regions and remain competitive with fixed baselines, but do not yield a significant improvement over the best fixed WSD schedule.

A first limitation is that the policy design still contains arbitrary choices. The 70% gate, the 10% lookback window, and the 3rd-percentile threshold were motivated by preliminary experiments and by the quantile-estimation argument in Appendix C, but they were not exhaustively tuned. These values could therefore be treated as hyperparameters in future work. In particular, a sliding lookback window updated throughout training may provide a cleaner and more local estimate of the current regime than a single threshold computed at the gate.

Section III-C2 also shows that WSD-PSGD does not improve performance (Figure 2). A plausible explanation is that AdamW's accumulated moment estimates are useful during decay: as the learning rate decreases, the adaptive second moment can help damp unstable or high-curvature directions. Removing this optimizer state may therefore make the descent less stable, even if the update appears closer to the simplified preconditioned-SGD intuition.

The correlation analysis in Section III-C3 should also be interpreted with caution. Many logged metrics are not independent, as discussed in Appendix B: for instance, `grad_weight_ratio`, `param_update_norm`, and `adam_v_norm` are all derived from overlapping gradient, weight, or optimizer-state quantities. Moreover, several metrics naturally evolve with training time, making it difficult to separate a true predictive signal from a proxy for optimization progress.

The choice of `loss_variance` and `grad_snr` as policy metrics is motivated in Appendix B. Nevertheless, Figure 4 suggests that this principled selection does not lead to a meaningful improvement over carefully chosen fixed schedules.

Overall, the main limitation of our method is that it does not yet provide a large enough improvement over strong fixed WSD baselines to clearly justify the added policy complexity. At our scale, a small grid over fixed WSD decay timings remains simple and very competitive [1]. At larger scale, the search space becomes much larger: decay timing, length, final LR ratio, metric choice, gate, and threshold can all interact. More compute and longer training runs may reveal more reliable patterns, but this must be balanced against the complexity added by the policy itself. Future work should tune these hyperparameters explicitly and explore sliding-window thresholds to make the trigger more adaptive to the current training regime.

VI. Summary

We study whether the decay phase of WSD learning-rate schedules can be triggered automatically from online training metrics. On a 40M-parameter LLaMA architecture model trained on FineWeb, we find that gradient SNR and loss variance carry useful signal about suitable decay regions, yet simple percentile-based policies do not significantly outperform a strong fixed WSD baseline. Metric-triggered decay is a promising direction, but its added complexity is only justified if future tuning yields clearer gains.

REFERENCES

[1] A. Hägele, D. Grangier, M. Ranzato, and N. Usunier, "Scaling laws and compute-optimal training beyond fixed training durations," *arXiv preprint arXiv:2405.18392*, 2024.

[2] I. Loshchilov and F. Hutter, "Sgdr: Stochastic gradient descent with warm restarts," *arXiv preprint arXiv:1608.03983*, 2016.

[3] J. Kaplan, S. McCandlish, T. Henighan, T. B. Brown, B. Chess, R. Child, S. Gray, A. Radford, J. Wu, and D. Amodei, "Scaling laws for neural language models," in *arXiv preprint arXiv:2001.08361*, 2020.

[4] S. Hu, Y. Tu, X. Han, C. He, G. Cui, X. Long, Z. Zheng, Y. Fang, Y. Huang, W. Zhao *et al.*, "MiniCPM: Unleashing the potential of small language models with scalable training strategies," *arXiv preprint arXiv:2404.06395*, 2024.

[5] K. Wen, Z. Li, J. Wang, D. Hall, P. Liang, and T. Ma, "Understanding warmup-stable-decay learning rates: A river valley loss landscape perspective," *arXiv preprint arXiv:2410.05192*, 2024.

[6] A. Defazio, A. Cutkosky, H. Mehta, and K. Mishchenko, "Optimal linear decay learning rate schedules and further refinements," *arXiv preprint arXiv:2310.07831*, 2023.

[7] A. Dubey, A. Jauhri, A. Pandey, A. Kadian, A. Al-Dahle, A. Letman *et al.*, "The llama 3 herd of models," *arXiv preprint arXiv:2407.21783*, 2024.

[8] A. Defazio, X. Yang, H. Mehta, K. Mishchenko, A. Khaled, and A. Cutkosky, "The road less scheduled," in *Advances in Neural Information Processing Systems*, 2024.

[9] T. Schaul, S. Zhang, and Y. LeCun, "No more pesky learning rates," in *Proceedings of the 30th International Conference on Machine Learning*, ser. Proceedings of Machine Learning Research, vol. 28. PMLR, 2013, pp. 343–351.

[10] G. Penedo, H. Kydlicek, L. B. allal, A. Lozhkov, M. Mitchell, C. Raffel, L. Von Werra, and T. Wolf, "The FineWeb datasets: Decanting the web for the finest text data at scale," *arXiv preprint arXiv:2406.17557*, 2024.

[11] "FineWeb dataset on HuggingFace," https://huggingface.co/datasets/HuggingFaceFW/fineweb.

[12] I. Loshchilov and F. Hutter, "Decoupled weight decay regularization," in *International Conference on Learning Representations*, 2019.

[13] A. W. van der Vaart, *Asymptotic Statistics*. Cambridge University Press, 1998.

APPENDIX

A. ADAMW AS MOMENTUM-FREE PRECONDITIONED SGD

The AdamW update at step t is

$$m_t = \beta_1 m_{t-1} + (1-\beta_1) g_t, \tag{1}$$

$$v_t = \beta_2 v_{t-1} + (1-\beta_2) g_t^{\odot 2}, \tag{2}$$

$$\hat{m}_t = \frac{m_t}{1-\beta_1^t}, \qquad \hat{v}_t = \frac{v_t}{1-\beta_2^t}, \tag{3}$$

$$\theta_t = \theta_{t-1} - \eta \frac{\hat{m}_t}{\sqrt{\hat{v}_t} + \varepsilon} - \eta\lambda\theta_{t-1}. \tag{4}$$

Here, g_t is the stochastic gradient, η is the learning rate, λ is the AdamW decoupled weight decay coefficient, m_t is the first-moment estimate, v_t is the second-moment estimate, and $\hat{m}_t$, $\hat{v}_t$ are their bias-corrected versions. The operation $g_t^{\odot 2}$, the square root, and the division are applied coordinate-wise.

Taking the limit $\beta_1 \to 0$ gives

$$m_t = g_t, \qquad \hat{m}_t = g_t.$$

The AdamW update therefore becomes

$$\theta_t = \theta_{t-1} - \eta \frac{g_t}{\sqrt{\hat{v}_t} + \varepsilon} - \eta\lambda\theta_{t-1}. \tag{5}$$

At the same time, $\beta_2 \to 1$ gives

$$v_t = \beta_2 v_{t-1} + (1-\beta_2) g_t^{\odot 2} \approx v_{t-1},$$

so v_t evolves slowly and retains long-term information about the coordinate-wise squared gradient magnitudes. The update is thus momentum-free because $\hat{m}_t = g_t$, but remains adaptively preconditioned through $\hat{v}_t$.

For comparison, SGD with decoupled weight decay is

$$\theta_t = \theta_{t-1} - \eta g_t - \eta\lambda\theta_{t-1}. \tag{6}$$

Thus, shrinking β_1 removes the first-moment momentum accumulated during the stable phase, while keeping β_2 close to 1 preserves the adaptive coordinate-wise scaling of AdamW. Under the river valley hypothesis of *Wen et al.* [5], this yields a momentum-free but preconditioned noisy SGD-like update, closer to the dynamics assumed in the theory than full AdamW with momentum.

B. ONLINE TRAINING METRICS

Table I lists the 10 diagnostic metrics recorded online during all WSD experiments. These metrics are computed at each logging step and track different aspects of training dynamics, including loss stability, gradient health, and parameter update magnitude.

Metric Dependencies and Redundancy: Several of the metrics in Table I are not fully independent. Some are derived from common underlying quantities and therefore provide partially overlapping information.

In particular, `grad_weight_ratio` combines `grad_norm` and `weight_norm`, making it a normalized measure of gradient magnitude relative to parameter scale. Similarly, `param_update_norm` is closely related to `grad_norm` through the optimizer update rule, although adaptive optimizers such as AdamW also introduce dependence on the historical second-moment estimate. The metric `adam_v_norm` is itself derived from a moving average of squared gradients and therefore reflects long-term gradient statistics.

TABLE I: Online diagnostic metrics recorded during training.

Metric	Definition
`loss_variance`	Variance of the loss over a sliding window of recent steps, measuring local stability.
`loss_oscillation`	Mean absolute difference between consecutive loss values, capturing short-term fluctuations.
`loss_improvement_rate`	Average per-step loss decrease over a recent window, measuring convergence speed.
`grad_norm`	ℓ_2 norm of the full gradient vector, $\|\nabla\mathcal{L}\|_2$.
`grad_snr`	Gradient signal-to-noise ratio, estimated as the ratio of the mean gradient to its standard deviation across parameters.
`grad_weight_ratio`	Ratio of gradient norm to weight norm, $\|\nabla\mathcal{L}\|_2 / \|W\|_2$.
`grad_cosine_sim`	Cosine similarity between gradients at consecutive steps.
`adam_v_norm`	ℓ_2 norm of Adam's second moment estimate v_t.
`weight_norm`	ℓ_2 norm of all trainable parameters, $\|W\|_2$.
`param_update_norm`	ℓ_2 norm of the parameter update ΔW at each step.

Metric Evolution Along Training: Several metrics also evolve naturally as optimization progresses. For example, loss-related quantities such as `loss_variance`, `loss_oscillation`, and `loss_improvement_rate` typically decrease as training approaches convergence, while gradient-based metrics often shrink as the optimizer moves toward flatter regions of the loss landscape.

Consequently, the most informative signals are often those that capture different aspects of training dynamics — such as loss behavior, gradient structure, and optimizer state — rather than multiple metrics derived from the same underlying quantity.

Correlation Analysis (Figure 5): Across all 10 metrics, the scatter plots reveal varying degrees of correlation with decay advantage. Several metrics like notably `grad_norm`, `grad_weight_ratio`, and `param_update_norm` display consistent per-ratio trends, but their slopes are largely driven by the shared dependence on raw gradient magnitude rather than an independent signal. Similarly, `adam_v_norm` tracks `grad_norm` closely, as it accumulates squared gradients over time and therefore reflects the same long-term gradient statistics. `weight_norm` and `grad_cosine_sim` show weaker or noisier trends, suggesting limited predictive value for decay timing.

Among all metrics, `grad_snr` and `loss_variance`

stand out as the most informative. `grad_snr` captures the consistency of the descent direction: a low SNR indicates that stochastic gradient noise dominates the signal, which under the river valley hypothesis [5] corresponds precisely to the regime where decay is most beneficial. `loss_variance` captures local loss stabilization: a drop in variance reflects that the model has settled into a flat region of the landscape and further progress under the stable learning rate is limited. Crucially, these two metrics are non-redundant because one reflects gradient structure while the other reflects loss behavior. This making their combination more informative than any pair of correlated metrics such as `grad_norm` and `param_update_norm`.

C. Variance of Percentile Estimation and Lookback Window Selection

The adaptive policies trigger decay when a metric falls below a low percentile of its recent history. Let q_p denote the true p-th quantile of the metric distribution and let $\hat{q}_p$ denote its empirical estimate computed from a lookback window containing n observations.

By the Central Limit Theorem for quantile estimators (Bahadur representation), the empirical quantile estimator is approximately normally distributed [13]:

$$\hat{q}_p \approx \mathcal{N}\left(q_p, \frac{p(1-p)}{nf(q_p)^2}\right), \tag{7}$$

where $f(q_p)$ is the density of the metric distribution evaluated at the true quantile. This result states that the quantile estimator becomes approximately Gaussian as the number of observations increases.

The corresponding variance is

$$\mathrm{Var}(\hat{q}_p) = \frac{p(1-p)}{nf(q_p)^2}, \tag{8}$$

and therefore

$$\mathrm{Var}(\hat{q}_p) = O(n^{-1}). \tag{9}$$

Thus, larger lookback windows produce more stable percentile estimates. The standard error is

$$\mathrm{SE}(\hat{q}_p) = \frac{\sqrt{p(1-p)}}{\sqrt{n}f(q_p)} = O(n^{-1/2}). \tag{10}$$

For the percentile used throughout this work,

$$p = 0.03, \tag{11}$$

which gives

$$\mathrm{Var}(\hat{q}_{0.03}) = \frac{0.03(1-0.03)}{nf(q_{0.03})^2} = \frac{0.0291}{nf(q_{0.03})^2}. \tag{12}$$

Thus, the threshold estimate stabilizes as the number of observations in the lookback window increases.

In our experiments, the lookback window is chosen as 10% of the total training budget T, so that

$$n = 0.1T. \tag{13}$$

Substituting this into the standard error scaling gives

$$\mathrm{SE}(\hat{q}_p) = O((0.1T)^{-1/2}) = O(T^{-1/2}). \tag{14}$$

This choice reduces the estimation error as the training budget increases while keeping the threshold based on a fixed fraction of recent observations. However, training metrics are non-stationary: if the lookback window is too large, the percentile estimate may include values from earlier optimization regimes and become less representative of the current training state. Increasing n therefore reduces estimation variance but also reduces locality.

The lookback window of 10% of the total training steps was chosen empirically as a practical compromise between these two effects. It provides enough observations for a stable percentile estimate while remaining sufficiently local to reflect the current phase of optimization.

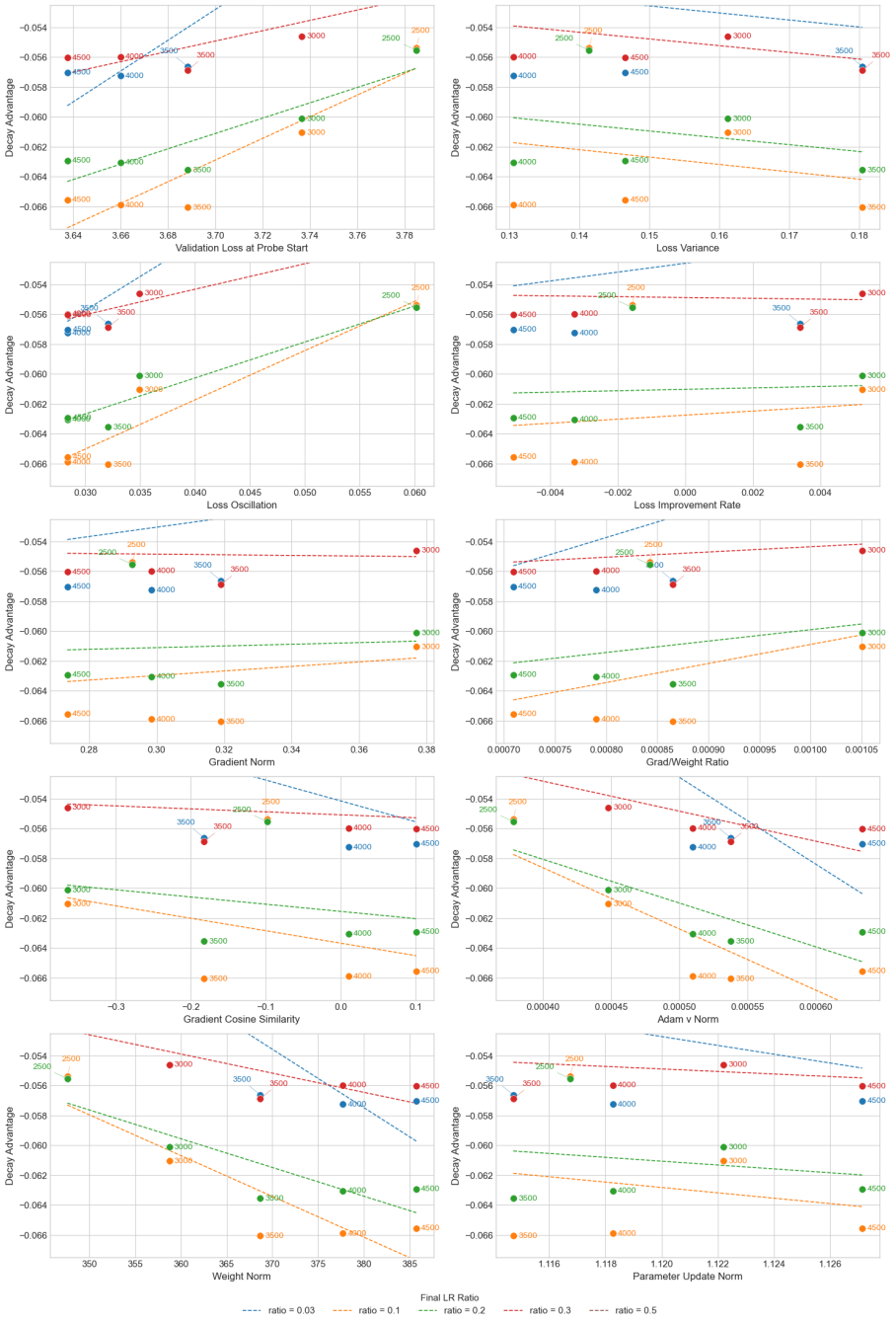

Fig. 5: Decay advantage vs. pre-decay diagnostic metrics for all 10 logged signals, across each combination of decay start step (point labels) and final LR ratio (color). Dashed lines show per-ratio linear trends. FineWeb, 5k steps total.